XML Interface Quick Reference Guide

Version 3

August 2026

EFT Corporation Services (Pvt) Limited

No part of this document may be reproduced or transmitted
In any form or by any means, electronic or mechanical,
For any purpose, without express written permission of:

The Managing Director
EFT Corporation Services (Pvt) Limited
100 The Chase, Emerald Hill
Harare, Zimbabwe

EFTCorp is committed to ongoing research and development to track technological developments and customers in the market. Consequently, information contained in this document may be subject to change without prior notice.

Document Version Control

Date	Version	Comments
20/01/2025	1	Initial Draft
14/03/2025	2	Updated to include change disbursement
26/08/2026	3	Added Appendix A (Acquirer identification via Structured Data) and Appendix B (VAS integration)

Table of Contents

1 INTRODUCTION 5

2 PREREQUISITES 5

3 XML MESSAGE FORMAT 5

4 INITIALIZING AND CLOSING 5

5 ERRORS 5

6 COMMUNICATIONS PROTOCOL 6

7 XML – TCP MESSAGES 6

8 XML – INITIALIZING A TERMINAL. 7

9 XML – CLOSING A TERMINAL. 7

10 XML – HANDLING TIMEOUTS. 8

11 XML – ERROR HANDLING. 10

11.1 OVERVIEW 10
11.2 ERROR MESSAGES 10

12 XML – SENDING A TRANSACTION 10

12.1 SINGLE MESSAGE PAIR MESSAGE FLOW 11
12.2 EVENTS AND CALLBACK BETWEEN THE POS DEVICE AND ESOCKET.POS 11
12.3 XML MAPPINGS IN A SINGLE MESSAGE PAIR PURCHASE TRANSACTION 12
12.4 EXAMPLE XML ELEMENTS AND ATTRIBUTES 13
12.5 EXAMPLE ZWG AND USD CURRENCY PURCHASE REQUEST XML MESSAGES: 15
12.6 EXAMPLE SUCCESSFUL ZWG CHIP CARD PURCHASE RESPONSE XML MESSAGE: 16

13 XML – EVENTS AND CALLBACK 18

13.1 XML - REGISTERING FOR EVENTS AND CALLBACK 18
13.2 XML - RECEIVING EVENTS AND CALLBACK 19
13.3 XML - SENDING EVENTS AND CALLBACK 20
13.4 XML – THE CASHBACK CALLBACK 21

14 XML – CHANGE DISBURSEMENT TRANSACTIONS 24

14.1 EXAMPLE CHANGE DISBURSEMENT XML MESSAGE: 25

15 APPENDIX A: ACQUIRER IDENTIFICATION VIA STRUCTURED DATA 26

15.1 EXAMPLE XML RESPONSE CONTAINING THE ACQUIRER KEY 26

16 APPENDIX B: VAS INTEGRATION (VALUE-ADDED SERVICES) 27

16.1 EXAMPLE VAS REQUEST AND RESPONSE 27
16.2 EXAMPLE VAS PROCURE AND RESPONSE 27

1 Introduction

This document provides an integration developer with a quick reference on how to initiate single message pair transactions using the XML Interface to enable the processing of EFT transactions through a PIN Pad. This solution is known as Intergraded POS or iPOS.

2 Prerequisites

- The development machine must have eSocket.POS software installed and configured.
- When using the VeriFone Vx805 or P200 PIN Pad, the development machine should have a free working USB Port and not have anything using the COM9 serial port. Some P200 devices require a free power socket for a power pack.
- An EFT Corporation Test Card
- Internet access on the development machine running eSocket.POS
- Access to the eSocket.POS developers guide document.

3 XML Message format

The XML interface is based on an XML format which closely mirrors the eSocket.POS API. Message types are represented using XML elements, and the fields within the message are represented as attributes. XML elements representing repeated data sets may also be contained within the message element. For example, a successful inquiry response may contain up to six ***Esp:Balance*** elements, which in turn contain attributes with the balance information. In an ***Esp:Transaction*** request, a single purchasing card element may be used to include attributes related to purchasing cards, and this element may in turn include repeated line item, tax amount and contact elements.

Request and response messages use the same XML element, unless an error occurs. Generally, the response message will contain attributes that are related to the response, for example ***ActionCode*** would appear only in a ***Esp:Transaction*** response message, and not in the request message.

4 Initializing and Closing

Before sending an EFT request message to the XML interface, an initialization message must be sent to register the terminal with eSocket.POS. When the terminal is finished operating, for instance at the end of the day, it can send a close message to eSocket.POS. These operations use an **Esp:Admin** request .

The ***Esp:Admin*** element takes the place of the ***init*** and close methods in the eSocket.POS API. There is no equivalent of the ***closeAll*** method.

5 Errors

If the XML interface is unable to process a message it receives, an ***Esp:Error*** response is returned. An example would be a message ***format error*** or a ***message received for an***

uninitialized terminal ID. If the XML interface can process the message, but an application error occurs within the eSocket.POS processing, a normal decline message will be returned of the same type as the request.

The ***Esp:Error*** element takes the place of the Java Exceptions that may be thrown when using the API.

6 Communications Protocol

Since messages are sent from the POS to eSocket.POS to initiate transaction processing, the message protocol must provide integrity in cases where message transmissions fail. Such failure may occur because of network connection failure or corrupted message data.

It is important to realize that any message transmissions from eSocket.POS to the Postilion Realtime engine are not part of this discussion, as eSocket.POS takes care of the message integrity on this leg. For example, if eSocket.POS sends a transaction request to Postilion Realtime, and does not receive a response, eSocket.POS will time out, respond negatively to the POS, and generate a reversal to Postilion Realtime. However, in the case where the POS sends a message to eSocket.POS, and does not receive a response from eSocket.POS, the POS must time out, and generate a reversal message to eSocket.POS.

Transaction requests and responses are communicated with eSocket.POS in XML format, sent using TCP with each message preceded by a length indicator. Please see below for additional information regarding TCP messages.

7 XML – TCP Messages

Communication between the POS and the XML interface uses a TCP message beginning with a header to indicate the length of the data segment of the message. This header must precede all messages sent to the XML interface and it will be sent at the beginning of all TCP responses from the XML interface.

If the message length is less than 2^{16} -1 or 65535 then a two-byte header will be used.

The first byte of the header contains the quotient of the length of the message (excluding this header) and 256. The second byte contains the remainder of this division. Both these values are represented in binary as unsigned integer values ranging from 0 to 255 (bytes 0x00 to 0xFF).

If the message length is greater than or equal to 2^{16} -1 or 65535 bytes then a six-byte header will be used.

In this case the bytes 0xFF 0xFF should be sent followed by a four-byte length indicator, i.e. six bytes in total.

8 XML – Initializing a terminal.

Before transactions can be sent from a terminal, the terminal must be initialized with eSocket.POS. This is done via the ***Esp:Admin*** element with the Type property set to '***INIT***'.

The **Terminal ID** identifies which terminal parameters to apply, so data for that Terminal ID must be present in the eSocket.POS database before that terminal can be initialized. The Terminal ID must be unique across all the terminals connecting to the upstream Postilion system.

In the case of a POS application controlling a single terminal, the initTerminal method should be called once as follows

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Admin TerminalId="Term1234" Action="INIT" />
</Esp:Interface>
```

To indicate that the terminal has been successfully initialized, a response of the following form will be returned:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Admin Action="INIT" ActionCode="APPROVE"
MessageReasonCode="9791"
TerminalId="Term1234"></Esp:Admin>
</Esp:Interface>
```

Note that the initialization process is memory and processor intensive and is typically called when the application starts, at the start of the day or after losing the TCP connection. **Terminals should not be initialized per transaction**.

9 XML – Closing a terminal.

When no more transactions are required, and a particular terminal is no longer needed, the POS application should close that terminal with eSocket.POS. This is done via the Esp:Admin element with the Type property set to 'CLOSE'.

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Admin TerminalId="Term1234" Action="CLOSE" />
</Esp:Interface>
```

To indicate that the terminal has been successfully closed, a response of the following form will be returned:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Admin Action="CLOSE" ActionCode="APPROVE"
MessageReasonCode="9791"
TerminalId="Term1234"></Esp:Admin>
</Esp:Interface>
```

10 XML – Handling Timeouts.

Once a transaction has been sent to eSocket.POS, the response may take a considerable time to be returned, as eSocket.POS may need to interact with the customer via external devices, and wait for an online response.

If no response is received within a reasonable time, the POS application needs to decide whether to ignore this, or to reverse the transaction, or send a repeat. **Note** that it is not necessary for the POS to handle a timeout if a valid response is received from eSocket.POS (even if the response code indicates a timeout), because in this case, eSocket.POS will take care of the repeat or reversal processing.

Message that timed out	Result
Esp:Inquiry	Ignore. Inquiries do not have financial impact so no reversal is required.
Esp:Transaction with *Type* of PURCHASE, DEPOSIT or REFUND, and *MessageType* not set or set to AUTH	Send a reversal.

Message that timed out	Result
Esp:Transaction with *Type* of ADMIN, CONFIRM or REFERRAL, or *MessageType* of CONFIRM, or *Reversal* set.	Send a repeat of the message. Repeats may be sent indefinitely (until a response is received from eSocket.POS), or limited in number, after which the transaction should be logged by the POS application for exception processing, for instance through an offline process.
Esp:Check with *MessageType* not set or set to TRANREQ	Ignore. Check verifications do not have financial impact so no reversal is required.
Esp:Check with *MessageType* of AUTHADV or CONFIRM	Send a repeat of the message. Refer to notes above about sending repeats indefinitely.
Esp:Merchandise with *Type* of REQUEST or PROCURE	Send a reversal. If the reversal response code indicates that the reversal was unsuccessful, it means the merchandise request could not be reversed. This may mean that the merchandise host regards the merchandise as dispensed and will expect payment for it. The steps to be taken in this situation are beyond the scope of eSocket.POS.
Esp:Merchandise with *Type* of REVERSAL	Send a repeat of the reversal. Refer to notes above about sending repeats indefinitely, and dealing with declined merchandise reversals.
Esp:Merchandise with *Type* of CONFIRM	Send a repeat of the message.
Esp:Admin	Send a repeat of the message.
Esp:Callback	Continue as if no ResponseData was returned.
Esp:Event	Not applicable: events use a fire-and-forget model, and no response is received.

11 XML – Error Handling.

11.1 Overview

When an error occurs and the XML interface is unable to process a message, an exception is raised and logged in the Java application, and an error response is returned to the POS application. Errors that may occur include:

- Attempting to set the property of an object to an invalid value
- An unsupported transaction type
- Attempting to send a message on behalf of a non-configured terminal
- A configuration error
- A timeout while waiting for a response to a request
- Errors during initialization

11.2 Error messages

Error messages use an Esp:Error element inside the Esp:Interface element, and take the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Error ActionCode="DECLINE"
MessageReasonCode="9791" ResponseCode="30"
TerminalId="Term1234" TransactionId="123456"></Esp:Error>
</Esp:Interface>
```

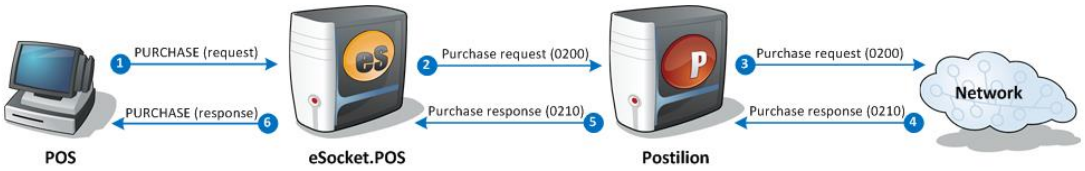
In some cases the error response may not include
the TerminalId and TransactionId attributes, for instance when an incoming XML message from the POS could not be parsed and these elements could not be extracted.

The error response can also contain a Description attribute, which contains a description of a software exception that occurred during the processing of the message. Typically this occurs when a terminal fails to initialize because the configuration is incorrect or a device is not available. The description contains an indication of the cause, and more complete details will usually be found in the eSocket.POS event log.

12 XML – Sending a transaction

This section discusses a single message pair purchase transaction, started at the POS and sent upstream via Postilion for authorization.

12.1 Single Message Pair Message Flow



Follow the message flow and refer to the diagram.

1. The POS sends a PURCHASE (request) XML message to eSocket.POS with the final amount of the transaction.
2. eSocket.POS sends a purchase request (0200) to Postilion.
3. Postilion sends the purchase request (0200) upstream.
4. Postilion receives a purchase response (0210) back from the entity upstream responsible for authorizing the transaction.
5. Postilion sends the purchase response (0210) to eSocket.POS.
6. eSocket.POS sends the appropriate response to the POS device.

Note: Events and callbacks are not displayed in the above diagram.

12.2 Events and Callback between the POS device and eSocket.POS

Events and callbacks are optional. Those listed here are a sub-set of the total list possible and are typical to this scenario.

If registered, the following events are called during a single message pair purchase transaction:

- PROMPT_INSERT_CARD
- PROMPT_SWIPE_CARD
- PROMPT_CONFIRM_TRAN
- PROMPT_PIN
- PROMPT_TRANSACTION_PROCESSING
- PROMPT_TRANSACTION_OUTCOME

If registered, the following callbacks are called during a single message pair purchase transaction:

- AUTHORIZE
- CARD_INFO
- CHECK_CARD_READER

- CONSECUTIVE_USAGE
- DATA_REQUIRED
- DEVICE_SERIAL_NR_AUTHORIZE

12.3 XML mappings in a single message pair purchase transaction

This section contains the XML properties and values that are typically associated with a single message pair purchase transaction. In the Value column, example values are in italics and mandatory values are in bold.

Property	Condition	Value
TerminalId	**M**	**TEST0001**
TransactionId	**M**	***195322***
Type	**M**	**PURCHASE**
TransactionAmount	G	*30000*
CurrencyCode	O	840

The **Terminal ID** is an 8-character alpha-numeric code unique to each terminal. It can be stored in the Retail Application Database and should be user configurable.

Transaction ID is a 6-digit number that uniquely identifies a transaction from this terminal within the eSocket.POS transaction retention period.

If a terminal sends more than one message for the same transaction (for instance, a request followed by a reversal), they should have the same transaction ID.

The transaction retention period is defined as a user parameter for the Transaction Cleaner component. A Transaction ID may only be reused once the cleaner has removed the original transaction from the database.

Transaction IDs beginning with zero should not be used. These are reserved for internal use by eSocket.POS.

Type defines the type of the transaction, which must be one of the following values:

The **TransactionAmoun**t is the amount of the transaction in minor denominations.

The **CurrencyCode** is the 3 digit currency code for the transaction's currency.

PURCHASE	Used to request a purchase of goods and services
DEPOSIT	Used to deposit funds to a cardholder's account
ADMIN	Used for administration advices, or for administration requests when the MessageType is set to 'ADMIN_REQUEST'.

12.4 Example XML elements and attributes

The eSocket.POS XML interface includes elements and attributes that that can be used to send various types of transactions to eSocket.POS. These transactions are based on the following elements:

- *Esp:Transaction*
- *Esp:Inquiry*
- *Esp:Merchandise*
- *Esp:Check*
- *Esp:Reconciliation*
- *Esp:Network*

A transaction message has the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
  <Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
    <Esp:Transaction CurrencyCode="924"
TerminalId="Term1234" TransactionAmount="2000"
TransactionId="123456" Type="PURCHASE"></Esp:Transaction>
  </Esp:Interface>
```

Other attributes may be set within the ***Esp:Transaction*** element depending on what is available on the POS.

The response from eSocket.POS may have the following form when using a chip card:

```
<?xml version="1.0" encoding="UTF-8"?>
  <Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
    <Esp:Transaction Account="10" ActionCode="APPROVE"
AmountTransactionFee="C0" AuthorizationNumber="248346"
AuthorizationProfile="11" BusinessDate="0408"
CardNumber="654123******0667"
CardProductName="DefaultCard" CardSequenceNumber="001"
CurrencyCode="932" DateTime="0407171329"
```

```
EmvAmount="000000002000" EmvAmountOther="000000000000"
EmvApplicationIdentifier="A0000007790000"
EmvApplicationInterchangeProfile="1800"
EmvApplicationTransactionCounter="00F1"
EmvAuthorizationResponseCode="00"
EmvCryptogram="BC969735FB8E69BB"
EmvCryptogramInformationData="40" EmvCvmResults="020300"
EmvIssuerActionCodeDefault="CC8834F800"
EmvIssuerActionCodeDenial="3070C80000"
EmvIssuerActionCodeOnline="CC8834F800"
EmvIssuerApplicationData="0FA501600000000000000000000000000
00F0100000000000000000000000000"
EmvTerminalCapabilities="E0F0C8"
EmvTerminalCountryCode="716" EmvTerminalType="22"
EmvTerminalVerificationResult="8080048000"
EmvTransactionCurrencyCode="932"
EmvTransactionDate="240407"
EmvTransactionStatusInformation="6800"
EmvTransactionType="00" EmvUnpredictableNumber="FDBE0562"
ExpiryDate="2805" LocalDate="0407" LocalTime="191329"
MerchantId="201000000000001" MessageReasonCode="9790"
PanEntryMode="05" PosCondition="00"
PosDataCode="91010151334C101" ResponseCode="00"
RetrievalRefNr="000000027884" TerminalId="52405177"
Track2="654123******0667=2805**************"
TransactionAmount="2000" TransactionId="241324"
Type="PURCHASE">
      <Esp:Balance AccountType="10" Amount="2000"
AmountType="53" CurrencyCode="932"
Sign="D"></Esp:Balance>
      <Esp:StructuredData DoNotPersist="FALSE"
Name="SrcLedgerBal" PersistUntilAuthorized="FALSE"
Value="000000000000"></Esp:StructuredData>
      <Esp:StructuredData DoNotPersist="FALSE"
Name="SrcAvailBal" PersistUntilAuthorized="FALSE"
Value="000000000000"></Esp:StructuredData>
      <Esp:StructuredData Name="ResponseFromPostilion"
Value="ResponseFromPostilion"></Esp:StructuredData>
      <Esp:StructuredData DoNotPersist="FALSE"
Name="SrcOdLimit" PersistUntilAuthorized="FALSE"
Value="000000000000"></Esp:StructuredData>
    </Esp:Transaction>
```

```
  </Esp:Interface>
```

The POS application may then extract the attributes in the response, for instance using the ActionCode to determine whether the transaction was approved. Other attributed might be printed on the receipt, saved to the POS database, etc.

12.5 Example ZWG and USD currency purchase request XML messages:

This is an example of a single message pair purchase transaction. The labels in bold and the numbers in square brackets refer to the corresponding labels and numbered arrows in the diagram in section 12.1.

The POS starts a single message pair purchase transaction with a PURCHASE (request) XML message [1] to eSocket.POS:

For Zimbabwe Gold (ZWG) Currency, you use currency code 924:

```
<?xml version="1.0" encoding="UTF-8"?>
  <Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Transaction
            CurrencyCode="924"
            TerminalId="TEST0001"
            TransactionAmount="1000"
            TransactionId="244035"
            Type="PURCHASE">
      </Esp:Transaction>
  </Esp:Interface>
```

For United States Dollars (USD) Currency, you use currency code 840:

```
<?xml version="1.0" encoding="UTF-8"?>
  <Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Transaction
            CurrencyCode="840"
            TerminalId=" TEST0001"
            TransactionAmount="1000"
            TransactionId="244035"
            Type="PURCHASE">
```

```
      </Esp:Transaction>
  </Esp:Interface>
```

12.6 Example successful ZWG Chip Card purchase response XML message:

The resulting PURCHASE (response) XML message (6) from eSocket.POS following the successful completion of a ZWG EMV Chip Card transaction will look like the message below:

```
<?xml version="1.0" encoding="UTF-8"?>
  <Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Transaction
            Account="10"
            ActionCode="APPROVE"
            AmountTransactionFee="C0"
            AuthorizationNumber="828517"
            AuthorizationProfile="11"
            BusinessDate="0510"
            CardNumber="654123******0667"
            CardProductName="DefaultCard"
            CardSequenceNumber="001"
            CurrencyCode="924"
            DateTime="0509044040"
      EmvAmount="000000001000"
      EmvAmountOther="000000000000"
EmvApplicationIdentifier="A0000007790000"
EmvApplicationInterchangeProfile="1800"
EmvApplicationTransactionCounter="00F7"
EmvAuthorizationResponseCode="00"
EmvCryptogram="FE850DEF31021E1D"
EmvCryptogramInformationData="40"
EmvCvmResults="020300"
EmvIssuerActionCodeDefault="CC8834F800"
EmvIssuerActionCodeDenial="3070C80000"
EmvIssuerActionCodeOnline="CC8834F800"
EmvIssuerApplicationData="0FA501600000000000000000000000
00F0100000000000000000000000000"
EmvTerminalCapabilities="E0F0C8"
EmvTerminalCountryCode="716"
```

```
EmvTerminalType="22"
EmvTerminalVerificationResult="8080048000"
EmvTransactionCurrencyCode="924"
EmvTransactionDate="240509"
EmvTransactionStatusInformation="6800"
EmvTransactionType="00"
EmvUnpredictableNumber="273E3148"
            ExpiryDate="2805"
            LocalDate="0509"
            LocalTime="064040"
            MerchantId="UAT000000000001"
            MessageReasonCode="9790"
            PanEntryMode="05"
            PosCondition="00"
            PosDataCode="A1010151334C101"
            ResponseCode="00"
            RetrievalRefNr="000000028603"
            TerminalId="TEST0001"
            Track2="654123******0667=2805**************"
            TransactionAmount="1000"
            TransactionId="244035"
            Type="PURCHASE">
      <Esp:Balance
            AccountType="10"
            Amount="1000"
            AmountType="53"
            CurrencyCode="924"
            Sign="D">
      </Esp:Balance>
      <Esp:StructuredData
            DoNotPersist="FALSE"
            Name="SrcLedgerBal"
       PersistUntilAuthorized="FALSE"
       Value="000009996500">
</Esp:StructuredData>
      <Esp:StructuredData
            DoNotPersist="FALSE"
            Name="SrcAvailBal"
            PersistUntilAuthorized="FALSE"
            Value="000009996500">
      </Esp:StructuredData>
      <Esp:StructuredData
            Name="ResponseFromPostilion"
```

```
            Value="ResponseFromPostilion">
      </Esp:StructuredData>
      <Esp:StructuredData
            DoNotPersist="FALSE"
            Name="SrcOdLimit"
            PersistUntilAuthorized="FALSE"
            Value="000000000000">
      </Esp:StructuredData>
    </Esp:Transaction>
  </Esp:Interface>
      </Esp:Interface>
```

13 XML – Events and callback

13.1 XML - Registering for events and callback

In order to receive events or callbacks from eSocket.POS, the POS application must register to receive particular events or callbacks for particular event IDs.

This is done in the initialization message for that terminal, by including one or more *Esp:Register* elements inside the initialization *Esp:Admin* message. This will take the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Admin TerminalId="Term1234"
Action="INIT"><Esp:Register Type="CALLBACK"
EventId="CALL_ID_1" />
            <Esp:Register Type="EVENT"
EventId="EVENT_ID_1" />
      </Esp:Admin>
</Esp:Interface>
```

See Terminal administration for a simple example of how to do this.

This means that the POS application will receive any events with the ID *EVENT_ID_1* or callbacks with the ID *CALL_ID_1* which might be sent by eSocket.POS. These will take the form of additional messages from the XML interface. Depending on which entities within eSocket.POS initiate these events or callbacks, they might occur at any stage during or outside normal transaction processing for an initialized terminal.

13.2 XML - Receiving events and callback

The POS application may register with the XML interface in order to receive events or callbacks with particular event IDs from eSocket.POS.

If it registers in this way, and eSocket.POS sends an event or callback with that event ID, the XML interface will send a message to the POS, using the *Esp:Event* or *Esp:Callback* element.

An event message will take the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Event TerminalId="TestTerm"
EventId="EVENT_ID_1" EventData="event data"/>
</Esp:Interface>
```

The POS may process the event in any way that it chooses, and no response should be sent.

A callback message will take the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Callback TerminalId="TestTerm"
EventId="CALL_ID_1" EventData="callback data"/>
</Esp:Interface>
```

The POS must return a response message to eSocket.POS, of the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Callback TerminalId="TestTerm"
EventId="CALL_ID_1" ResponseData="response data" />
```

```
</Esp:Interface>
```

The POS application may process the callback any way it chooses, but it must return a response containing the *ResponseData* attribute. If no response data is available, this should be indicated by an empty string inside quotes: *ResponseData="".* The response may require a certain amount of time, particularly if the return value requires customer or operator interaction. eSocket.POS allows a maximum of 60 seconds, after which an empty return value is assumed.

If the POS does not want to process an event which it receives, it is free to ignore it. If the POS does not want to process a callback which it receives, it must still return a response.

13.3 XML - sending events and callback

The eSocket.POS XML interface provides a mechanism to allow the POS application to send events or callbacks to eSocket.POS. These take the form of additional messages using the *Esp:Event* or *Esp:Callback* elements inside the main *Esp:Interface* element.

Events use a fire-and-forget model, so the POS will not be aware of whether any internal entity in eSocket.POS has registered to receive the event, or what the results of receiving it were. Event messages sent to eSocket.POS take the following form:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Event TerminalId="Term1234"
EventId="POS_EVENT" EventData="data"/>
</Esp:Interface>
```

Callbacks are handled within eSocket.POS by whatever entity has registered to receive the callback. Callbacks are sent as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Callback TerminalId="Term1234"
EventId="POS_CALLBACK" EventData="pos callback data" />
</Esp:Interface>
```

Callbacks may take some time to return, for instance if customer interaction is required via an external device. A callback return message is sent as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
      <Esp:Callback TerminalId="Term1234"
EventId="POS_CALLBACK" ResponseData="response data"/>
</Esp:Interface>
```

If no entity has registered, or no response data is available, the empty string *ResponseData=""* will be returned.

Whether these events or callbacks are handled by eSocket.POS depends on internal entities of eSocket.POS being registered for the particular event IDs used. This will be based on the configuration and customization of eSocket.POS, and should be determined during the integration phase between eSocket.POS and the POS application.

13.4 XML – The Cashback Callback

After registering for the cashback event, you should get cashback callbacks which you can respond to.

Please see example below of the response which you can make to the callback event if there is no cashback for your transaction.

eSocket.POS will send the following callback.

```
<?xml version="1.0" encoding="UTF-8"?>

<Esp:Interface Version="1.0"
xmlns:Esp=http://www.mosaicsoftware.com/Postilion/eSocket
.POS/>

<Esp:Callback TerminalId="TERM1234"
EventId="DATA_REQUIRED"
EventData="CashbackAmount(50000)"/>

</Esp:Interface>
```

Your response should be a zero if no cashback is required.

```
<?xml version="1.0" encoding="UTF-8"?>

<Esp:Interface Version="1.0"
xmlns:Esp=http://www.mosaicsoftware.com/Postilion/eSocket
.POS/>

<Esp:Callback EventId="DATA_REQUIRED"
ResponseData="0"></Esp:Callback>

</Esp:Interface>
```

The POS application may process the callback any way it chooses, but it must return a response containing the ResponseData attribute. If no response data is available, this should be indicated by an empty string inside quotes: ***ResponseData="".*** The response may require a certain amount of time, particularly if the return value requires customer or operator interaction. eSocket.POS allows a maximum of 60 seconds, after which an empty return value is assumed.

If the POS does not want to process an event which it receives, it is free to ignore it. If the POS does not want to process a callback which it receives, it must still return a response.

The cashback callback is used to insert a cashback amount to a transaction. You initiate a normal purchase transaction with no cashback. The callback will then ask you if cashback is required. If you confirm that cashback is required, it should then prompt you for the cashback amount.

Below is an example Init Message example which registers for the callback ***<Esp:Register Type="CALLBACK" EventId="DATA_REQUIRED"/>:***

```
..<?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
<Esp:Admin TerminalId="TERM1234" Action="INIT">
```

```
<Esp:Register Type="CALLBACK" EventId="DATA_REQUIRED"/>
</Esp:Admin>
</Esp:Interface>
```

After the terminal initializes, you can send transactions to eSocket.POS. See example of a typical transaction flow below;

Send transaction as normal:

```
. <?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
<Esp:Transaction RetrievalRefNr="100000000072"
TerminalId="TERM1234" TransactionAmount="000000042900"
TransactionId="100007" Type="PURCHASE"/>
</Esp:Interface>
```

You will get an Event to insert card. You may or may not display its EventID to the customer:

```
<?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
<Esp:Event TerminalId="TERM1234"
EventId="PROMPT_INSERT_CARD"
EventData=""/></Esp:Interface>
```

After eSocket.POS collects the PAN, it will then prompt for the cashback amount:

```
<?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
```

```
<Esp:Callback TerminalId="TERM1234"
EventId="DATA_REQUIRED"
EventData="CashbackAmount(50000)"/>
</Esp:Interface>
```

You can respond to that callback with something like the following for a 200.00 cashback value:

```
..<?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
<Esp:Callback ResponseData="20000"
EventId="DATA_REQUIRED" TerminalId="TERM1234"/>
</Esp:Interface>
```

Or if cashback is not required for the transaction, you can respond with something like the following:

```
..<?xml version="1.0" encoding="UTF-8"?><Esp:Interface
Version="1"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocke
t.POS/">
<Esp:Callback ResponseData="0" EventId="DATA_REQUIRED"
TerminalId="TERM1234"/>
</Esp:Interface>
```

After this, the next message which we will need is the transaction outcome XML message.

14 XML – Change Disbursement Transactions

Change disbursement transactions make use of the deposit transaction type to enable a retail application to deposit a customer's change due to their mobile wallet or bank card if supported. At the conclusion of a cash sale, if there is change due, the cashier will be presented with an option to "Deposit Change" to the customer's electronic wallet. Below is an example of the XML message to be sent by selecting this option.

14.1 Example change disbursement XML message:

```
<?xml version=\"1.0\" encoding=\"UTF-8\"?><Esp:Interface
Version=\"1.0\"
xmlns:Esp=\"http://www.mosaicsoftware.com/Postilion/eSocket.POS/\">
<Esp:Transaction CurrencyCode="{CURRENCY_CODE}"
TerminalId="{TERMINAL_ID}"
TransactionAmount="{CHANGE_AMOUNT}"
TransactionId="{TRANSACTION_ID}" Type="DEPOSIT"
ExtendedTransactionType="6001"></Esp:Transaction></Esp:In
terface>
```

Where:

{CURRENCY_CODE}" = 840 (since the functionality is meant for USD Cash Transactions only)

{TERMINAL_ID} = the terminal ID in use for the till

{CHANGE_AMOUNT} = the amount of USD change owed to the customer in minor denominations

{TRANSACTION_ID} = 6-digit transaction number generated from the POS

Type = Hardcoded to DEPOSIT

ExtendedTransactionType = Hardcoded to 6001 to facilitate in correct routing on Postilion Realtime System upstream.

You can also include a Retrieval Reference Number (RRN) if your retail application generates retrieval reference numbers for purchase transactions. (OPTIONAL)

15 Appendix A: Acquirer Identification via Structured Data

eSocket.POS enriches every authorisation response (Esp:Transaction) with the identity of the acquiring bank before it reaches the retail application. This is delivered as an additional Esp:StructuredData entry, keyed "Acquirer", alongside any other Esp:StructuredData already returned on that message.

- The Acquirer key appears only on authorisation responses; it is never present on request (outbound) messages.
- Locate it by filtering the Esp:StructuredData children of Esp:Transaction for Name="Acquirer" - the order of Esp:StructuredData entries is not fixed, so do not match by position.
- The Value attribute is variable-length, alphanumeric, up to 25 characters (for example "FCB").
- Acquirer is populated on both approved and declined responses; code the lookup defensively (null-safe) even though the key is expected on every response.

15.1 Example XML response containing the Acquirer key

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocket.POS/">
<Esp:Transaction MerchantId="FCBUAT000000001" TerminalId="FCBUAT01"
 ResponseCode="05" ActionCode="DECLINE" Type="PURCHASE" ... >
<Esp:StructuredData Name="ResponseFromPostilion"
Value="ResponseFromPostilion"></Esp:StructuredData>
<Esp:StructuredData DoNotPersist="FALSE" Name="Acquirer"
 PersistUntilAuthorized="FALSE" Value="FCB"></Esp:StructuredData>
</Esp:Transaction></Esp:Interface>
```

Where:

Name = the fixed literal "Acquirer" - always match on this attribute, not on element position.

Value = the acquiring bank identity for this transaction (e.g. "FCB"); treat as an opaque string, not an enumerated field.

16 Appendix B: VAS Integration (Value-Added Services)

Value-Added Services (VAS) - ZESA prepaid electricity tokens, airtime, water and bill payments - use a different message element to standard card purchases: Esp:Merchandise (not Esp:Transaction). This appendix summarises the message flow only; for the complete specification (field classification, currency handling, token decoding, receipt printing, and security considerations) refer to the companion VAS Integration Developer Guide.

- A VAS purchase is a three-phase exchange over the same TCP session used for INIT: REQUEST (validates the meter/account and amount), PROCURE (commits the purchase and returns the token or narrative result), and REVERSE (reverses a PROCURE that must be backed out).
- Only ProductType=ELECTRICITY (ZESA) is implemented in the current release.
- REVERSE must only be sent against a purchase that has already been PROCUREd - never against a REQUEST that was never PROCUREd.

16.1 Example VAS REQUEST and response

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocket.POS/">
<Esp:Merchandise DateTime="0807143205" ExtendedTransactionType="9000"
ProductId="80000129"
 ProductType="ELECTRICITY" RetrievalRefNr="260807143205" TerminalId="TEST0001"
 TransactionAmount="100" TransactionId="614532" UserId="263773176741"
Type="REQUEST">
<Esp:PosStructuredData Name="MeterNumber" Value="14316800680"/>
</Esp:Merchandise></Esp:Interface>
```

Successful response returns the account holder's name and billing currency for the cashier to confirm before proceeding to PROCURE:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocket.POS/">
<Esp:Merchandise ActionCode="APPROVE" ResponseCode="00"
RetrievalRefNr="260807143205"
 TerminalId="TEST0001" TransactionAmount="100" Type="REQUEST">
<Esp:StructuredData Name="customerName" Value="J MOYO"/>
<Esp:StructuredData Name="meterCurrency" Value="USD"/>
</Esp:Merchandise></Esp:Interface>
```

16.2 Example VAS PROCURE and response

Commits the purchase using the same RRN, TransactionId, ProductId, ProductType, UserId, MeterNumber and TransactionAmount as the REQUEST (a fresh STAN and DateTime are generated for this message):

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocket.POS/">
<Esp:Merchandise DateTime="0807143207" ExtendedTransactionType="9000"
ProductId="80000129"
 ProductType="ELECTRICITY" RetrievalRefNr="260807143205" TerminalId="TEST0001"
 TransactionAmount="100" TransactionId="614532" UserId="263773176741"
Type="PROCURE">
<Esp:PosStructuredData Name="MeterNumber" Value="14316800680"/>
<Esp:PosStructuredData Name="MerchantName" Value="ZETDC"/>
<Esp:PosStructuredData Name="ProductName" Value="ZETDC_PREPAID"/>
<Esp:PosStructuredData Name="ApiProcessingCode" Value="U50000"/>
<Esp:PosStructuredData Name="CurrencyCode" Value="840"/>
</Esp:Merchandise></Esp:Interface>
```

Successful response - the Token field is what the POS must decode and print:

```
<?xml version="1.0" encoding="UTF-8"?>
<Esp:Interface Version="1.0"
xmlns:Esp="http://www.mosaicsoftware.com/Postilion/eSocket.POS/">
<Esp:Merchandise ActionCode="APPROVE" ResponseCode="00"
RetrievalRefNr="260807143205"
 TerminalId="TEST0001" TransactionAmount="100" LocalDate="0807" LocalTime="143209"
Type="PROCURE">
<Esp:StructuredData Name="Token"
Value="12345678901234567890|5.23|0.1912|123456789|85.00|15.00|0.15"/>
</Esp:Merchandise></Esp:Interface>
```

Non-token products (TELEPHONE PREPAY, WATER, BILL_PAYMENT) typically return Esp:StructuredData Name="Narrative" instead of "Token" - a free-text confirmation string to print in place of a token block. Always check for Token first and fall back to Narrative.

For prerequisites, the hardcoded/configurable/dynamic field classification, STAN/RRN generation rules, currency handling, the full token decode format, response code 09 (asynchronous "in progress"), receipt printing, and security considerations specific to VAS, refer to the VAS Integration Developer Guide (companion document).